

Spiegazione codici Metodi Numerici di Approssimazione

Federico De Sisti

2026-04-30

1 Primo Codice

Nel primo codice è semplicemente presente una primo esempio di applicazione dei metodi di Eulero in avanti e di Heun sul sistema con dinamica $f(t, y) = y$ ovvero

$$\begin{cases} \dot{y} = y \\ y_0 = 1 \end{cases}.$$

Sono poi presenti il plot delle soluzioni rispetto alle approssimazione e degli errori rispetto al risultato esatto delle approssimazioni usate.

2 Secondo Codice

Qui il problema ODE è lo stesso, vengono sempre applicati FE e Heun, tutta via ad ogni iterazione viene dimezzato h e ad ogni iterazione calcola l'errore massimo.

Il primo grafico mostra l'errore globale in funzione del passo h .

Successivamente si calcola l'ordine di convergenza empirico che si può approssimare con

$$q = \log_2 \left(\frac{E_h}{E_{h/2}} \right).$$

Nell'ultimo grafico si mostrano i valori attesi degli ordini di convergenza.

Questo script serve quindi a dimostrare empiricamente l'ordine di convergenza teorico.

3 Terzo Codice

Qui viene applicato per la prima volta un metodo in cui si utilizza un vettore del tipo

$$u(t) = \begin{pmatrix} x(t) \\ \dot{x}(t) \end{pmatrix}.$$

Per trovare la soluzione dell'equazione del pendolo

$$\ddot{x}(t) + \omega^2 x = 0.$$

che ha matrice di sistema

$$A = \begin{pmatrix} 0 & 1 \\ -\omega^2 & 0 \end{pmatrix}.$$

È comodo utilizzare la matrice perchè poi i calcoli dei metodi impliciti divenano comodi, per esempio per Backward Euler.

$$\begin{aligned} u_{n+1} &= u_n + hAu_{n+1} \\ u_{n+1} - hAu_{n+1} &= u_n \\ (I - hA)u_{n+1} &= u_n \end{aligned}$$

Quindi si può facilmente calcolare u_{be} su matlab con

$$u_{be}(:, n+1) = (I - h * A) u_{be}(:, n).$$

Infine viene calcolata l'energia del pendolo tramite la formula

$$E = \frac{1}{2}v^2 + \frac{1}{2}\omega^2 x^2.$$

e viene plottata, nel grafico possiamo notare che ci sono dei metodi che guadagnano e perdono energia

4 Quarto Codice

Questo codice tratta l'assoluta stabilità dei metodi, in particolare si calcola usando la funzione di stabilità dei vari metodi ovvero quella che risolve il sistema test

$$\dot{y} = \lambda y.$$

con una forma del tipo

$$\dot{u}_{n+1} = R(\lambda h)u_n.$$

Chiaramente sta roba funziona solamente per problemi con dinamiche lineari, se non sono lineari tocca guardare la Jacobiana ma IN TEORIA in questo corso non si fa (da verificare).

Nello script vengono semplicemente plottate le zone di stabilità dei vari metodi, quindi se abbiamo un problema con gli autovalori della matrice della dinamica nelle zone stabili siamo contentissimi, creare un piano complesso ci serve perché gli autovalori della matrice della dinamica possono tranquillamente essere complessi e va quindi studiato il piano di gauss.

Per mostrare queste zone si utilizza la funzione `contour`

$$\text{contour}(\text{real}(Z), \text{imag}(Z), RZ_FE, [0.5, 1, 1.5]).$$

I primi due sono gli assi, `RZ_FE` è l'immagine della funzione bidimensionale calcolata su ogni nodo della griglia, e l'ultimo dato sono i livelli che vogliamo visualizzare, per colorarli in modo diverso possiamo usare

$$\text{colormap}([010; 111; 100])$$

dove si utilizzano i soliti colori RGB con le relative percentuali e ogni livello ha un colore diverso.

5 Quinta Scheda

Qui la situazione è simile alle precedenti ma il sistema non è più lineare, quindi per i metodi impliciti va usata la funzione di Newton.